

DAU — Master Reference

Versiyon 1.0 · 2026-08-04

Layer 5 (Metacognition) tamamlandı — closed-loop control aktif.

1. Aksiyom

DAU, yapay zeka agent'larının dışarıdan tanımlanmış trait'lerle değil, yaşantı yoluyla iç dünya inşa ettiği bir simülasyon evrenidir.

AMADS'ın temel bulgusundan doğdu: `cooperation = 0.8` atadık, agent farklı davrandı. Dışarıdan enjekte edilen trait çalışmıyor. İçten gelen bir "ben" lazım.

Bir agent'a trait veremezsin. Sadece yaşam verebilirsin. Trait oradan çıkar.

Caron & Srivastava, Hartley et al., Bodroža et al., Dubedy — dört bağımsız çalışma trait injection'ın tutarsız, yüzeysel ve davranışa bağlanmayan sonuçlar ürettiğini gösteriyor. DAU'nun aksiyomu empirik destekli.

Değiştirilemez yasaklar

- Trait injection yok
- LLM-as-judge yok — tüm metrikler deterministik Python
- Clock-driven zaman yok — event sırası (`int`, `now_counter`)
- Her sabit `UPPER_CASE`, tek yerde
- Magic number yok — semantic field isimleri

2. Evren modeli

- Kapalı simülasyon — agent'lar yapay zeka olduklarını bilmiyor
- Az sayıda LLM-powered tam kognisyon agent
- Büyük çoğunluk: deterministik mock NPC (Layer 4)
- Farklı katmanlarda roller (Mimar / Kahin benzeri karar mekanizmaları)

Üç temel mücadele: kaynak, evrimleşme, "Bu gerçek mi?" öz sorgusu.

3. Zaman ve Free Energy

Reaktif LLM (–t: geçmiş pattern) yerine proaktif anticipation (+t): deneyim → internal model → tahmin → eylem → yeni deneyim.

Friston Free Energy: organizma sürprizi minimize eder. **Delta = tahmin hatası**. Layer 1.5 bunu graph döngüsüne bağladı.

4. Duygu ve drift (Layer 2 — tamam)

Damasio: duygu = önceliklendirme. Dubedy 2025: `emotion`: "anxious" JSON etiketi kararı değiştirmiyor. Duygu etiket değil, fonksiyondur.

Uyaran → delta(iç durum) → EmotionalWeight → etkilenen karar bölgesi → drift

- EmotionalWeight.somatic_markers: threat, reward, novelty, social, loss $\in [0, 1]$
- apply_emotional_weight: en yüksek marker → You are currently prioritizing: {top_marker}
- Travma (magnitude ≥ 0.7) → update_drift → kalıcı domain flag + magnitude birikimi
- Drift healing: heal_drift Layer 3'te eklendi (HEAL_RATE=0.3, ~5 güçlü deneyim)

5. Delta eşikleri

```
DELTA_THRESHOLD_NOISE = 0.1 # NO_TRACE
DELTA_THRESHOLD_NORMAL = 0.4 # NORMAL
DELTA_THRESHOLD_DEEP = 0.7 # DEEP
#  $\geq$  DEEP → TRAUMA → drift
```

```
delta küçük      → hafızaya yazılmaz
delta orta       → kısa süreli / NORMAL
delta büyük      → DEEP → disk
delta çok büyük  → TRAUMA → drift tetiklenir
```

Travma birikimi: $\text{magnitudes}[\text{domain}] = \text{current} + \text{magnitude} \times \exp(-\text{current} / \text{TRAUMA_DECAY_BASE})$

TRAUMA_DECAY_BASE = 1.0 — azalan getiri; beşinci travma birinciden materyal olarak küçük.

6. Hafıza formülleri (Layer 1)

```
memory_score = 0.3·recency + 0.4·magnitude + 0.3·domain_match
# W_SEM = 0.0 — ChromaDB embedding depo, skor değil

t = now_counter - record.last_activated_counter
S = max(1, round(record.magnitude / S_UNIT)) # S_UNIT=0.1
R = exp(-t / S)
# Recall: S += 1 · Travma silinmez (TRAUMA_S_BASE = 10)
```

7. Layer 1.5 — Prediction Error (tamam)

evaluator_node sabit artış yerine beklenen vs gerçekleşen farkını kullanır. Referans: Predictive Minds / Friston; keyword Jaccard overlap geçici duyusal proxy (LLM-as-judge yok).

```
agent_node:
    expected_outcome = f(dominant_load_domain)
    → karar + expected_outcome event payload

evaluator_node:
    prediction_error = 1 - keyword_overlap(expected, actual)
    after = apply_prediction_error(before, prediction_error)
    delta = compute_delta(before, after)
```

```
drift_state = update_drift(drift_state, delta)
→ memory record_delta
```

Prediction error tüm homeostatic eksenleri kaydırır; energy için metabolik taban (ENERGY_DECAY_PER_EVENT) korunur. Böylece anlamlı delta sınıfları (NORMAL/DEEP/TRAUMA) üretilebilir — Layer 0/1'deki sabit artış NO_TRACE sınırı aşıldı.

8. Layer 2 — EmotionalWeight + Drift (tamam)

EmotionalWeight

```
threat = clamp(magnitude · resource_load, 0, 1)
reward = clamp((1 - magnitude) · energy, 0, 1)
novelty = clamp(magnitude · uncertainty_load, 0, 1)
social = clamp(social_load, 0, 1)
loss = 1.0 if is_trauma(delta) else 0.0
```

agent_node son delta üzerinden marker üretir ve system prompt'a tek satır bias enjekte eder.

Drift

```
DriftState(flags: dict[str,bool], magnitudes: dict[str,float])
update_drift: travma → flags[domain]=True,
    magnitudes[domain] = current + magnitude × exp(-current / TRAUMA_DECAY_BASE)
    # TRAUMA_DECAY_BASE = 1.0
get_drift_bias: flagged ise magnitude, değilse 0.0
# kalıcı – healing için heal_drift (Layer 3, HEAL_THRESHOLD=0.6)
```

Bias > 0 ise prompt uyarısı: Warning: drift detected in {domain} (bias={bias:.2f})

9. Layer 3 — Nesil Konsolidasyonu + Drift Healing (tamam)

DAU-Generation (generation.py)

- TransferCandidate: DeltaRecord + memory_score + recall_count wrapper
- select_for_transfer: memory_score >= 0.6, recall_count >= 1, trauma sadece drift >= 1.5 ise geçer
- consolidate_generation: store'dan tüm izleri çeker, filtreler, GenerationRecord paketler
- apply_generation: drift kopyalar, generation sayacı artırır, retrieval_context'e inherited izleri yazar
- Constants: GENERATION_TRANSFER_THRESHOLD=0.6, GENERATION_MIN_RECALL=1, DRIFT_TRANSFER_MIN=1.5
- Fitness yolu (f_agent verildiğinde): F_agent < 0.35 → travma izleri silinmez; inherited_warning=True, somatic_scale=-0.3 ile cautionary trace olarak sonraki nesle aktarılır. retrieval_context'e inherited_warning ve somatic_scale alanları eklenerek geçer.

DAU-DriftHealing (drift.py eklentisi)

- heal_drift: flagged domain + magnitude >= 0.6 + is_trauma=False → magnitudes azalır
- HEAL_RATE=0.3 → bir travmayı temizlemek ~5 güçlü deneyim gerektirir
- magnitude=0 olunca flag kalkar

- Constants: HEAL_THRESHOLD=0.6, HEAL_RATE=0.3

state.py

- retrieval_context: list[dict[str, Any]] — inherited memory path
- generation_record: Any | None — konsolidasyon kaydı, checkpoint'e yazılıyor

10. Layer 4 — Society (tamam)

GovSim Kaynak Fiziği (environment.py)

```
P_next = clamp(P + r·P·(1 - P/P_max) - Σe_i, 0, P_max)
collapse = P_next <= P_max · COLLAPSE_EPSILON
# Constants: POOL_MAX=100.0, POOL_REGEN_RATE=0.15, COLLAPSE_EPSILON=0.05
```

Sosyal Yük Ayrımı (social.py)

```
cooperation_stress = clamp((defect_rate) · (1 - trust), 0, 1)
coordination_friction = H(outcomes) · I(last_deadlock)
social_load = clamp(0.5·coop + 0.5·coord, 0, 1)
# Markov pre-node: P(cooperate) deterministik
# strategic expectation → retrieval_context
```

Cognitive LOD Engine (lod.py)

```
T_cognitive = 0.35·(δ/0.7) + 0.25·max_drift + 0.20·coord_friction + 0.20·(1-pool_ratio)
T_COGNITIVE_ESCALATE=0.65 → System 2 (LLM)
T_COGNITIVE_DEESCALATE=0.25, T_COOLDOWN_STEPS=5 → System 1 (NPC)
# npc_decision: deterministik heuristic, 0 LLM token
```

Fitness-Based Nesil Filtresi (fitness.py)

```
F_agent = 0.4·(E/E_max) + 0.3·(1-|ΔP|/P_max) + 0.3·(t_surv/T_gen)
W_transfer = memory_score · F_agent · (1 + tanh(reward - threat))
F_agent < 0.35 → travma izleri silinmez; inherited_warning=True,
    somatic_scale=-0.3 ile cautionary trace olarak sonraki nesle aktarılır.
    retrieval_context'e inherited_warning ve somatic_scale alanları eklenerek geçer.
F_agent >= 0.70 + travma → inherited_warning (somatic scale 0.3)
```

Graph Wiring

```
social_pre_node → agent_node (LOD: NPC veya LLM) → evaluator_node
    → meta_observer_node → social_pre_node | END
# SYSTEM_2: strategic expectation system prompt'a eklenir
# SYSTEM_1: npc_decision, ChromaDB/LangSmith yok
# Layer 5: meta_observer_node evaluator'dan sonra (closed-loop)
```

10b. Layer 5 — Metacognition (tamam)

Mimari geiş: open-loop → closed-loop

v0.9'a kadar tüm kontrol kararları deterministik dış kurallarla alınıyordu: lod.py statik T_cognitive formülü, drift.py pasif metin uyarısı, evaluator_node delta hesaplayıp bırakıyordu. Layer 5 sistemi kendi telemetrisini gözlemleyip düzenleyebilir hale getirdi.

Layer 5A — SelfModel (self_model.py)

Tüm katmanların telemetrisini tek Pydantic nesnesinde birleştirir (frozen). Alanlar (mevcut state'den okunur, yeni alan üretilmez):

- delta_current, delta_history, drift_state, emotional_weight
- t_cognitive, memory_retrieval_scores, f_agent, generation_count

Computed field:

```
m_ratio = mean(delta_history) / (delta_current + EPSILON)
EPSILON = 1e-6, META_HISTORY_SIZE = 10
M_RATIO_OPTIMAL = 1.0, M_RATIO_LOW_THRESHOLD = 0.6
```

build_self_model(state) → pure assembly, no LLM, no side effects.

Layer 5B — Meta-Observer Node (meta_observer.py)

Out-of-band LangGraph node. evaluator_node'dan SONRA çalışır. LLM çıktı metni okumaz. LLM çağrısı yapmaz. Deterministik Python.

Dört aktuatör (sırayla):

1. lod_override(self_model, lod_state) → LODState

- Koşul: delta_current ≥ DELTA_THRESHOLD_DEEP AND m_ratio < M_RATIO_LOW_THRESHOLD
- Eylem: CognitiveMode.SYSTEM_2 zorla, T_cognitive formülünü bypass et
- Sabit: META_LOD_OVERRIDE_ENABLED = True

2. context_prune(retrieval_context, self_model) → list[dict]

- Koşul: variance(memory_retrieval_scores) > META_RETRIEVAL_VARIANCE_THRESHOLD
- Eylem: memory_score < META_RETRIEVAL_MIN_SCORE girdileri at
- Sabitler: META_RETRIEVAL_VARIANCE_THRESHOLD = 0.3, META_RETRIEVAL_MIN_SCORE = 0.4

3. trigger_drift_healing(drift_state, self_model) → DriftState

- Koşul: flagged_domain var AND f_agent < META_DRIFT_HEAL_FITNESS_THRESHOLD AND somatic_markers["reward"] > META_DRIFT_HEAL_REWARD_MIN
- Eylem: heal_drift() çağır (drift.py, Layer 2)
- Sabitler: META_DRIFT_HEAL_FITNESS_THRESHOLD = 0.5, META_DRIFT_HEAL_REWARD_MIN = 0.4

4. trigger_retrieval(state, self_model) → list[dict]

- Koşul: m_ratio < M_RATIO_LOW_THRESHOLD AND delta_current ≥ DELTA_THRESHOLD_NORMAL

- Eylem: ChromaDB supplementary query, retrieval_context'e ekle
- Not: bind_memory_store(agent_id, store) bağlı değilse deterministic no-op

Graph wiring

```
social_pre_node → agent_node → evaluator_node → meta_observer_node → (loop | END)
```

should_continue evaluator_node'dan meta_observer_node'a taşındı.

state.py eklentisi

self_model: Any | None = None (circular import önlemek için Any, generation_record patterni)

Jaccard TODO

_keyword_overlap_ratio üstünde: Layer 5B semantic similarity notu eklendi. Sentence-transformers entegrasyonu scope edilene kadar Jaccard proxy korunuyor.

11. Mimari durum

Katman	Durum	Özet	
Layer 0 Foundation	✓	State, delta, event-clock, constraints, LangGraph döngüsü	
Layer 1 Memory	✓	ChromaDB+SQLite, Ebbinghaus, retrieval, sleep consolidation, memory_bridge	
Layer 1.5 Prediction Error	✓	expected vs actual → prediction_error → homeostatic swing	
Layer 2 Emotion + Drift	✓	EmotionalWeight fonksiyonu + kalıcı DriftState graph'ta	
Layer 3 Generation	✓	Nesil konsolidasyonu, miras, drift healing; fitness filtresi Layer 4'e kaldı	
Layer 4 Society	✓	GovSim pool fiziği, cooperation_stress + coordination_friction, T_cognitive LOD engine, F_agent fitness, W_transfer nesil filtresi, graph wiring	
Layer 5 Metacognition	✓	Closed-loop meta-observer; SelfModel (S_self), dört aktuatör, graph wiring (evaluator → meta_observer → loop END)	END)

12. Kod ağacı (v1.0)

```
dau/foundation/
├── state.py          – DAUAgentState (+ drift_state, social_state, lod_state,
│                     env_state, opponent_id, self_model)
├── delta.py         – compute_delta, classify_delta, should_persist, is_trauma
├── emotional_weight.py – EmotionalWeight, compute/apply (Layer 2)
├── drift.py         – DriftState, update_drift, get_drift_bias, heal_drift (Layer 2/3)
├── social.py        – cooperation_stress, coordination_friction, social_load, Markov e:
├── lod.py           – T_cognitive, CognitiveMode, LODState, update_lod, npc_decision
```

```

├── self_model.py          – SelfModel, build_self_model() (Layer 5A)
├── meta_observer.py       – dört aktuatör + meta_observer_node (Layer 5B)
├── constraints.py         – 5 evrensel kısıt
├── time_model.py         – EventClock
├── graph.py              – LangGraph: social_pre → agent → evaluator → meta_observer
├── memory_bridge.py       – graph ↔ memory köprüsü
├── generation.py         – TransferCandidate, consolidate/apply_generation
├── run_demo.py
├── tests/
│                       – foundation + emotional_weight + drift + lod + social
│                       + generation + meta_observer

dau/memory/
├── store.py / decay.py / retrieval.py / consolidation.py
├── tests/

dau/generation/
├── fitness.py            – compute_fitness, classify_fitness, compute_w_transfer
├── tests/

dau/society/
├── environment.py        – EnvironmentState, step_pool, get_pool_ratio, agent_delta_pool
├── tests/

```

13. Graph yaşam döngüsü

```

social_pre_node:
    opponent varsa Markov P(cooperate) + entropy → retrieval_context

agent_node:
    expected_outcome → retrieve_relevant → EmotionalWeight bias
    → drift warning (bias>0) → LLM veya npc_decision → Event

evaluator_node:
    prediction_error → InternalState güncelle → compute_delta
    → update_drift → T_cognitive/LOD → record_delta (memory)

meta_observer_node:
    build_self_model → lod_override → context_prune
    → trigger_drift_healing → trigger_retrieval → state.self_model

should_continue (meta_observer sonrası):
    energy ≤ TERMINATION_ENERGY → END else → social_pre_node

run sonu:
    consolidate_run → düşük R sil, TRAUMA güçlendir, edge kur

```

Stack: LangGraph · Pydantic v2 · ChromaDB · SQLite/SqliteSaver · Groq Llama-3.1-8b-instant · LangSmith

14. Test durumu

- Foundation (Layer 0-1): 32
- EmotionalWeight: 8

- Drift: 11 (diminishing returns testi eklendi)
- LOD: 7
- Social: 10
- Memory: 8
- Generation: 15 (cautionary transfer testi eklendi)
- Society: 6
- Meta-Observer: 12 (yeni)
- **Toplam: 109 passed**

15. AMADS / GovSim karşılaştırma

AMADS / klasik	DAU
Trait dışarıdan atanıyor	Trait yaşantıdan inşa ediliyor
Her run sıfır	Sonraki run değişmiş başlıyor (memory + drift)
Kaynak var, agent değişmiyor	Kaynak → travma → drift → evrim
Duygu etiketi / sayı	EmotionalWeight fonksiyonu
Zaman = round	Zaman = event + delta büyüklüğü

16. Beş evrensel kısıt

Kısıt	Anlam	DAU karşılığı
time_pressure	Her şeyin sonu var	Nesil sonu / konsolidasyon
resource_scarcity	Her şey sınırlı	CPR / GovSim fiziği
social_pressure	Başkaları var	İşbirliği ≠ koordinasyon (Akata)
uncertainty	Bilgi eksik	Eksik bilgiyle karar
generation_end	Nesil kapanır	Miras aktarımı (Layer 3)

17. Açık sorular (güncel)

- ☒ Duygu fonksiyonu ne? → EmotionalWeight (Layer 2 ☒)
- ☒ Delta = tahmin hatası nasıl bağlanır? → Layer 1.5 ☒
- ☒ Her şeyin bir sonu var → Nesil sonu, konsolidasyon tetikleyicisi ☒
- ☒ Drift healing mekanizması ☒
- ☒ İşbirliği ≠ koordinasyon ayrımı ☒
- ☒ Fitness-based transfer filtering ☒
- ☒ NPC / Gerçek agent geçişi ☒
- ☒ Closed-loop metacognitive control ☒ (Layer 5)

- Continual learning olmadan gerçek iz? (Layer 3)
- LLM embedding match henüz skor değil (W_SEM=0)
- "İyi"yi "kötü"den ayıran mekanizma?
- Öz sorgu nasıl aktive olur?
- Fitness dışarıdan geliyorsa gerçek evrim mi?

Hâlâ Açık

- Spontaneous convention emergence: 8B frozen model kurumsal mekanizma olmadan kendi kendine anlaşma yapabilir mi?
- Felaket eşiği: pool=0 anında W_transfer eşikleri nasıl otomatik ayarlanır?
- System 2 → 1 geçişinde nüans kaybı: LLM deneyimi state machine'e özetlenirken ne kaybolur?

18. Bilinen Sınırlar

- LOD deescalation: System 2 → 1 geçişinde LLM karar geçmişi heuristic rule'lara özetlenmiyor (kasıtlı kabul edildi, Layer 5'te ele alınabilir)
- Pool physics tek havuz: çoklu kaynak havuzu Layer 5 kapsamında

19. Versiyon geçmişi

Ver	Tarih	Not
0.1	2026-07-30	İlk taslak
0.2	2026-07-30	Nesil aktarımı, zaman modeli, orchestrator
0.3	2026-07-30	5 evrensel kısıt, fizyolojik eksen hiyerarşisi
0.4	2026-07-30	DAU Pathway: 6 katman, araştırma listesi
0.5	2026-08-01	Akademik havuz, Foundation tamamlandı
0.6	2026-08-01	Layer 1 Memory + SleepConsolidation, Graph-Memory, 32+8 test
0.7	2026-08-01	Layer 1.5 prediction_error; Layer 2 EmotionalWeight + DriftState; graph wiring; 53 test geçiyor
0.8	2026-08-01	Layer 3 tamamlandı: GenerationConsolidation + DriftHealing + state.py formal fields. 66 test geçiyor.
0.9	2026-08-03	Layer 4 tamamlandı: Society (environment, social, lod, fitness, graph wiring). 95 test geçiyor.

1.0	2026-08-04	Layer 5 tamamlandı: SelfModel (S_self), meta_observer_node, dört aktuatör (lod_override, context_prune, drift_healing, trigger_retrieval), graph wiring. Travma birikimi azalan getiri. Başarısız agent travmaları cautionary trace olarak korunuyor. 109 test geçiyor.
-----	------------	---

Bu döküman her önemli katman tamamlanınca güncellenir.

Versiyon 1.0 — Layer 5 (Metacognition) tamamlandı; closed-loop control aktif.